Blocaïne

User manual

(The open-source solution for automation)

Table of contents

- 1 - Generality

Blocaïne is a software platform designed for automation of industrial processes. It comprises two main components:

- **Blocks editor**,Who allow you to create programs by assembling functional blocks.
- **Interpreter,** Who executes programs created with the **Block editor,** thus controlling industrial equipment via remote inputs/outputs.

The **Block editor** is installed on a “Development PC» And communicates via Ethernet with the Executor deployed on another computer called the "Target Machine".

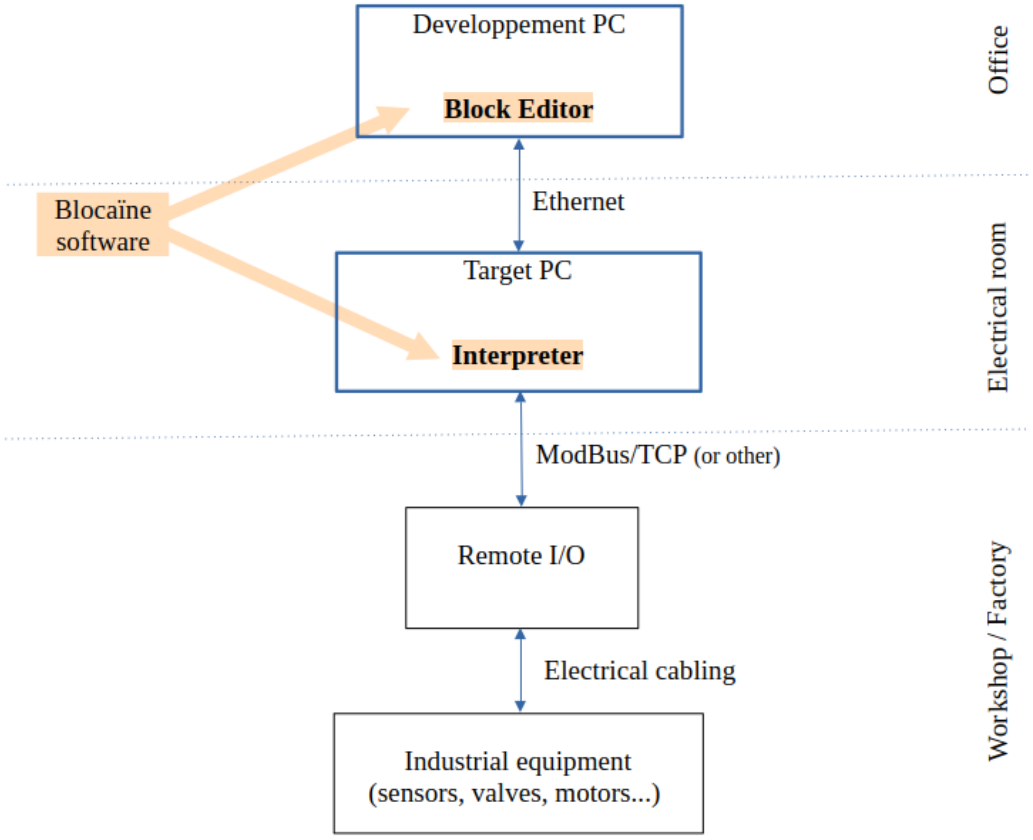


Figure 1 : General Architecture

The Ethernet connection allows programs created with the **Block Editor** to be transferred to the **Interpreter** it also provides real-time visualization in the **Block Editor** of variables from programs currently running in the **Interpreter**.

- 1.1 - Features

- **"Hot Swap”**: A new version of a program can be loaded while the old version is currently running.
- Graphical programming in the form of linked functional blocks
- Each variable natively has a validity bit, which propagates from block to block.
- Dynamic typing of variables.
- The order in which the blocks are executed is managed automatically.
- The layout of the blocks is done automatically
- A web server allows to know the status of the Target
- OPC UA allows interface with IHM/SCADA
- ModBus/TCP allows interface with remote I/O
- Blocaïne is cross-platform (Linux, Windows, macOS...)
- Blocaïne is royalty-free and open source

- 1.2 - Project progress

The Blocaïne project is at the functional prototype stage.

You can download it, for evaluation on "github.com",The installation procedure is described in the document "evaluation_en.pdf»

For more information, you can contact me by email: blocaine@barachet.com

- 1.3 - Precautions for uses, responsibilities

The use of Blocaïne is at the user's own risk.

- 2 - Principles

- 2.1 - Everything is block

Each block consists of input(s), output(s), and the code that evaluates the outputs based on the inputs.

- For the basic blocks (called **System** blocks) it's a function written in Python that evaluates the outputs based on the inputs.
- For user-created blocks (called **user** blocks) it's a chain of **system** blocks or **user blocks** which allows us to evaluate the outputs based on the inputs

An executable program is simply a **user** block, a subroutine also take the form of a **user** block, the inputs/outputs of the blocks are themselves blocks (<inputs>,<output>), the <input> block has an output defined by an <output> block, while the block <output> has an output defined by a <input> block. In sort, everything is a block.

- 2.2 - Hot Swap

The **interpreter** has two versions for each program (**user** block) : Shift A and Shift B. The new version of a block is always downloaded into the Shift that is not currently running. When a Hot Swap is initiated, the current values of the stored variables from the Shift that is currently running are transferred to the other Shift, which then takes over...

- 2.3 - Method of execution

For each running **user** block, the **interpreter** periodically evaluates all the <output> blocks associated with a periodic task. the input of an <output> block is evaluated by the **interpreter** by first executing the previous block, inputs of this previous block is evaluated by first executing previous block and so on.

This approach eliminates the need to explicitly define the execution order of the blocks.

- 2.4 - Typing of variables

Since Blocaïne is based on the Python language, the variables (input and output) used by the blocks are dynamically typed; their type can be anything, including complex structures, as long as it is a combination of types recognized by Python (string, integer, float, list, dictionary, etc.).

- 2.5 - Validity bit

A validity bit associated with each block output is automatically evaluated at each execution, depending on the validity of the inputs and the possibility or impossibility of evaluating the output.

- 2.6 - Default values

Each block entrance, whether it is a s**ystem** or **user** block has a default value that can be modified at instantiation.

- 3 - Hardware

Given that Blocaïne consists of only two Python programs (one for the **Interpreter** and one for the **Block Editor**), the hardware only needs to be able to run Python (version 3.6 or higher) code and to have a network adapter (Internet connection), either Wi-Fi or wired, for **Blocaïne** to operate.

- 3.1 - OS and “real-time”

All operating systems are, in principle, compatible (Windows, Linux, macOS, etc.), but their real-time performance is not identical.
Only the **Interpreter** is subject to real-time constraints; the **Block Editor,** which serves solely as an operator interface, can run under any OS.
You can assess the stability of periodic tasks through the web server provided by the target (the Interpreter) by comparing the theoretical period with the minimum and maximum cycle times on the “Task & Load” page.
After many tests, it can be said that a “standard” Linux OS is sufficient in most cases, but if the real-time constraints are demanding, a switch to Linux-RT or another real-time solution will be necessary.

- 3.2 - Hardware for Evaluation

A simple Windows PC, Linux PC, or Mac is perfectly suitable, since the version downloadable from GitHub.com (see evaluation procedure) is configured to run both the **Interpreter** and the **Block Editor** on the same machine.

- 3.3 - Hardware for Standard Application

- 3.3.1 - Development PC

Nowadays, Python is natively present on “all” PCs and Macs. Since the **Block Editor** uses only Python standard modules, installing it reduces to a simple file copy. Your favorite laptop is therefore perfectly suitable for running the **Block Editor,** as long as Python version 3.6 or higher is installed.

- 3.3.2 - Target Machine

The Raspberry Pi 5 appears to be the best choice for running the **Interpreter** for the following reasons:

- **Price**: around €50 for the 1 GB RAM version
- **Single-core performance**: thanks to its Arm Cortex-A76
- **Ease of deployment**: by downloading an ISO image for the SD card, which avoids installing non-standard Python modules

- 4 - Block editor

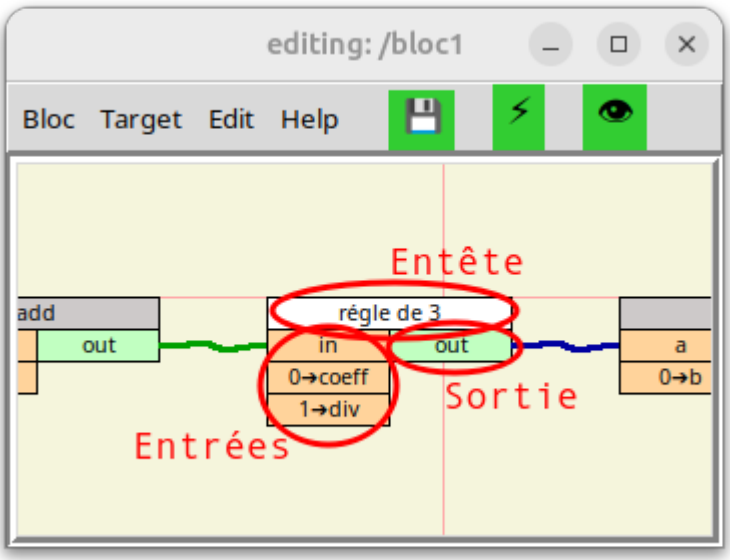
- 4.1 - Generalities

The **Block editor** allows :

- Creating and modifying programs in form **user** block containing functional blocks chains between them.
- Compiling, downloading and launching the current **user** block.
- Monitoring (Dynamic visualization) of I/O values of blocks.
- Overriding of I/O values of blocks.

Each block consists of:

- A header (Entête): containing the block name
- The entry(ies) (Entrées): located(s) left below the heading, containing the name of the entry.
- The exit(s) (Sortie): located(s) right under the heading, containing the name of the output.



Blocks can be chained together by links, note that an output can only be linked to an input and an input can only be linked to a single output.

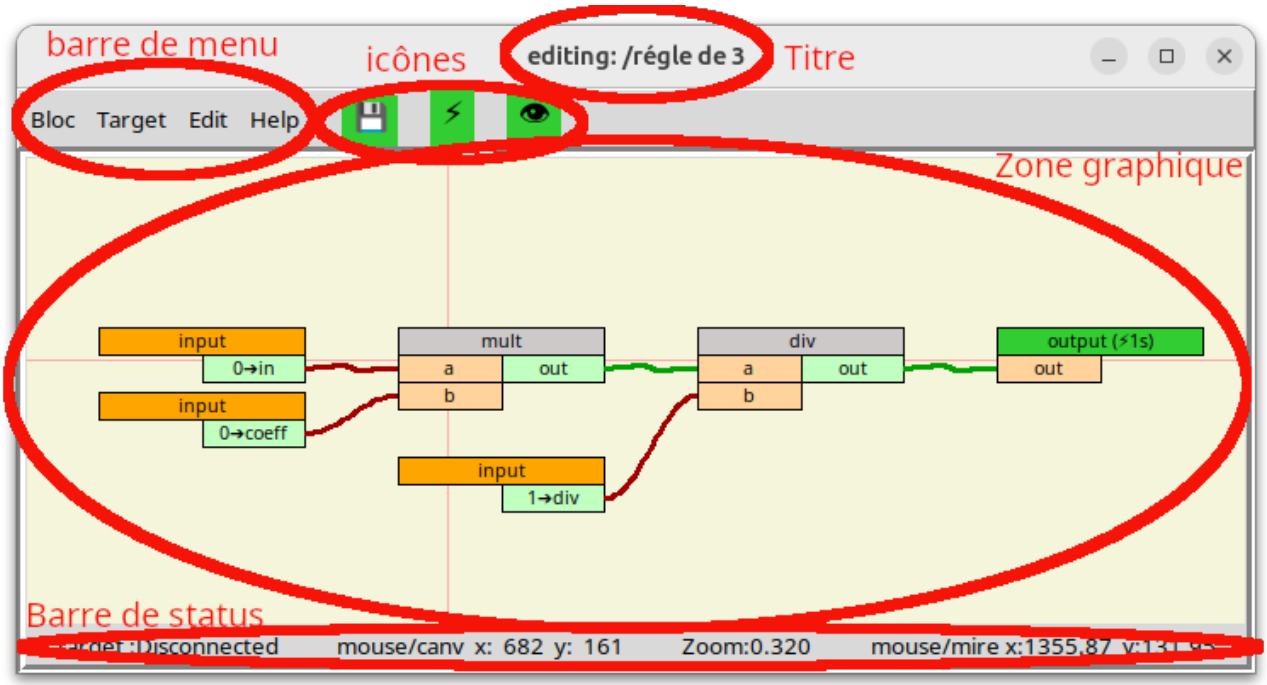
Each block corresponds to a file named: " *block name*.bloc ". **System** block files

are located in the /blocs/system directory, while the **user** block files are in /blocs/user.

- 4.2 - Graphical interface

The graphical interface of the "Blocaïne" **block editor** consists of one or more windows, each containing the following
elements:

1. A title bar : (Titre)
2. A menu bar : (barre de menu):
3. Three icons 💾, ⚡,👁 : (icônes)
4. A graphic area : (Zone graphique)
5. A status bar: (Barre de status)



Graphical interface of the **block editor**

- 4.2.1 - Title bar

At the top of each window is a title bar which contains:

- current mode: **Editing** or **Monitoring**
- the name of the **user** block currently being edited or debugged

Note that if the block was opened as a subblock, sub-subblock, etc., the path used to open it will appear before the block name. This is useful in**Monitoring (**debugging) mode to determine which instance of the block variables displayed in this window belong to.

- 4.2.2 - Menu bar

The menu bar located in the top left corner below the title bar contains the following four menus:

- **Bloc**: is equivalent to the "File menu" in most softwares.
- **Target**: to manage the interface with the "Target" (so with the **Interpreter).**
- **Edit**: to manage the presentation.
- **Help**: documentation.

- 4.2.2.a - "Bloc" menu:

- **Open**: to open a file containing a block.
- **Save [F3]**: to save the block being edited in a file whose name corresponds to the name of the current block.
- **Save as [F4]**: to save the block being edited to a file whose name will be entered by the user.
- **Update [F5]**: to update all the external interfaces of the blocks used as a subroutine in the currently being edited **user** block.
- **Change Properties:** to modify the header fields associated with the **user** block currently being edited, such as "the author of the block", "keywords"...but also its "name".
- **Open new windows**: Opens a new **Block Editor** in a separate window, fully independent from the original **Block Editor**.
- **Quit [Escape]**: quits the current window, without saving the **user** block **c**urrently being edited.

- 4.2.2.b - "Target" menu:

- **Disconnect from Target**Or**Connect to Target:**to connect to or disconnect from the "Target Machine"(so at the**Executor**).
- **Check <*block_name*>**: to check if the block being edited is valid and compilable.
- **Check& Transferrt <*block_name*>**: checks that the block being edited is valid, compiles it, then transfers the result to the "Target" to the **Shift** that is not "running".
- **Check, Transferrt & Execute <*block_name*>**: checks that the block being edited is valid, compiles it, transfers the result in the "Target" to the **Shift** that is not "running", then the Interpreter launches its execution: in the event that the block is already running on the other **Shift**, a **Hot Swap** will be initiated. Otherwise a simple start (Run) will be initiated.
- **Monitoring (Start/Stop) [Space]**: allows switching between edit mode and debug mode.

- 4.2.2.c - "Edit" menu:

- **Auto layout [F7]:**This command performs an automatic layout.
- **Status bar (show/hide):**This command allows you to show or hide the "status bar".

- 4.2.2.d - "Help" menu:

- **Versions**: gives the version of the **Block editor** and the block structure.
- **Mouse usage**: gives instructions on how to use the mouse:
  - [Left click] to select, to use menu
  - [Left held down] to scroll within window, to move block, to link I/O
  - [Left double-click] to open **user** block
  - [Right button] to open the context menu
  - [wheel button] to monitor the target variables (on/off)
  - [wheel rotation] to zoom (in/out)
- **Key use**: gives instructions on use the keyboard:
  - [F1] Open the documentation for the block whose header is located under the cursor
  - [F3] to save current block
  - [F4] to save current block as
  - [F5] to update current block (update sub-block interfaces)
  - [F7] to layout current block
  - [Delete] Delete the block whose header is located under the cursor
  - [Space] to monitor the target variables (on/off)
  - [Escape] to close current **Block Editor** window
- **General Documentation**: opens this document.

- 4.2.3 - Icons

These icons provide quick access to the most frequently used features:

- "💾": Save the current block.
- "⚡": Checks the consistency of the current block, converts him (compiles it) in a format understandable by the Interpre**ter**, transfer the result to the “Target”, then start its execution:
  1. If the current block is not already running in the “Target”, a start order (run) is initiate.
  2. If the current block is already running in the “Target”, a **Hot Swap** order is initiate.
- "👁️": Switching between Editing and Monitoring modes.

- 4.2.4 - Graphics Area

This area allows you to build a program (**user** block) by depositing blocks (**system** or **user)** and by linking them together.

- 4.2.4.a - Navigation

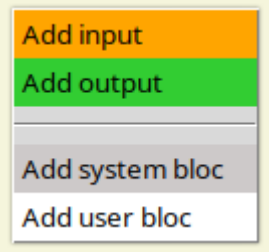
To navigate this area, you need to use a mouse :

- **Rotating the well mouse**: allows you to zoom in/out.
- **Mouse movement with the left button held down**:
  - **On empty area:** allows to move the visible area (scroll)
  - **On I/O:** allows you to create a link between an input and an output
  - **On block header:** allows to move the block
- **Right-click**: allows you to bring up the context menu.
- **Double left click :**
  - **On user block header**: opens the **user** block pointed in a new window.
  - **On an I/O of a user block**: opens the **user** block of the pointed I/O in a new window and positions the input or the output in the center of the graphics area
- **Left click**: allows selected a field in a menu.
- **Click on the scroll wheel button**: Enables/disables real-time visualization of variable values located in the target (Monitoring On/Off)

- 4.2.4.b - Context menus

4.2.4.b.1 Default Menu

A Right-click on a virgin area, displays following context menu:

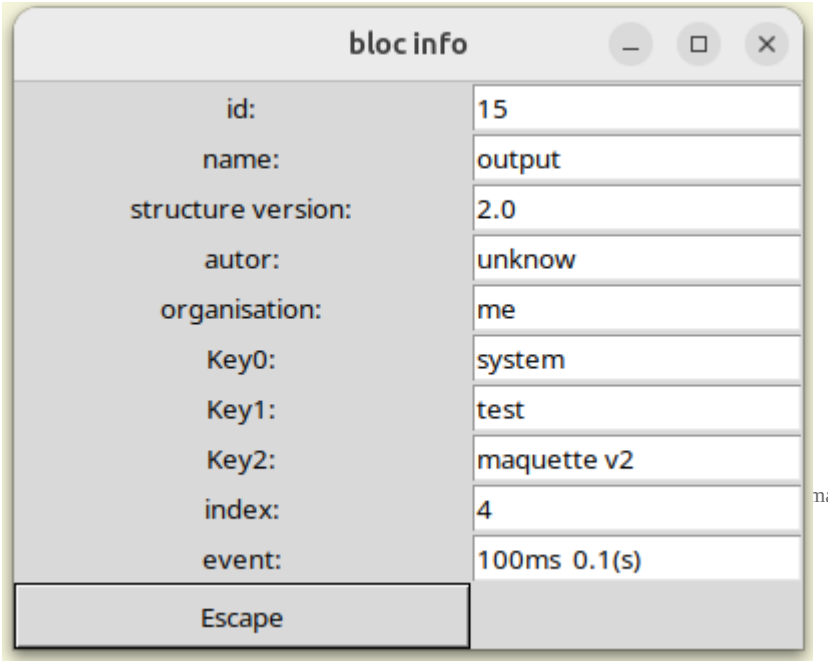


This menu allows you to add a block.

4.2.4.b.2 “header” Menu

A Right-click on the header of a block, displays the following context menu:

1. **Delete**: allows the block to be deleted
2. **Update**: updates the block's external interface, from that block's file.
3. **Change event** : allows the block to be associated with a periodic task so that it can be executed. This field is only available for <output> blocks.
4. **Position: 2/3**, **move up**,**move down**: allows you to change the order of inputs or outputs in the external interface of a **user block c**urrently being edited. This field is only available for <input> or <output> blocks.
5. **Documentation**: opens the block help file in the default web browser. This field is only available if a help file named "block_{*type of block}_{name of block}*.html » is located in the “/Documentation” directory.
    1. *Type of block* : maybe; “**user”** or **“system”**
    2. *name of block* **:** corresponding to the name defined in the block header
6. **Properties**: displays the block properties like this.

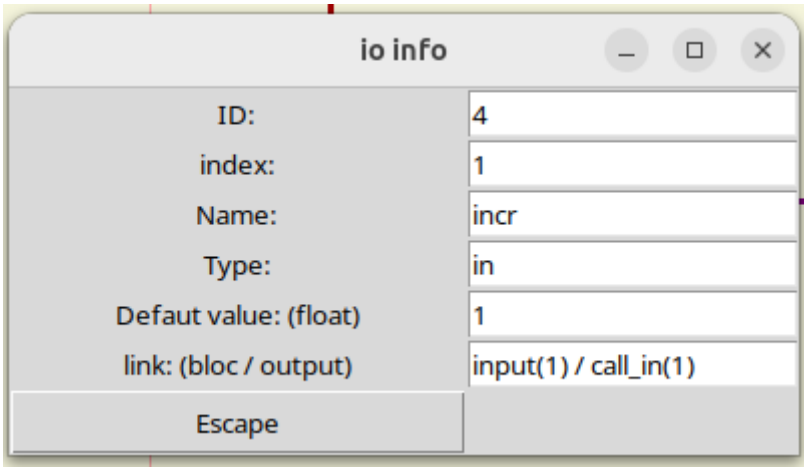


4.2.4.b.3 “Input/Output” Menu

A right-click on a input or output of a block, displays he following context menu:

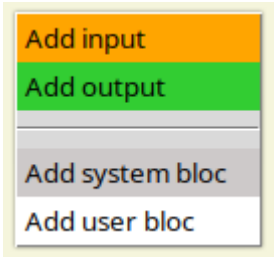
1. **Delete link** : Allows removing the link with the previous block. This field is only available if you clicked on an input connected to an output.
2. **Rename:** allows modifying the name of an input or an output; this new name is associate with the block currently being edited, the new name will appear in the block's external interface. This field is only available for <input> or <output> blocks.
3. **Change comment:** allows modifying the comment of an input or an output; this new comment is associate with the block currently being edited, the new comment will appear in the block's external interface. This field is only available for <input> or <output> blocks.
4. **Set default value**: allows to give default type and value to the input, if it is an <input> block the default value will appear in the block’s external interface; otherwise it is associated with the block instance.
5. **Open**: allows opening a new window that edits the **user** block of the input/output, and positions the input/output in the center of the graphics area.
6. **Create locale name**: allows defining the name associated with the block’s instance. This field is available for all blocks except <input> and <output>.
7. **Delete locale name**: allows removing name associated with the block’s instance. This field is available for all blocks except <input> and <output>.
8. **Create local comment**: allows defining comment associated with the block’s instance. This field is available for all blocks except <input> and <output>.
9. **Delete local comment**: allows removing comment associated with the block’s instance. This field is available for all blocks except <input> and <output>.
10. **Add Memory** : Allows defining an output as a “stored” output. This field is only available for outputs of a **system** block, i.e., for the input of an <output> block when the block currently being edited is a **system** block.
11. **Delete Memory** : Allows removing the “storage” of an output. This field is only available for “stored” outputs of a **system** block, i.e., for the input of an <output> block when the block currently being edited is a **system** block.
12. **Add initial value** : Allows assigning an initial value associated with the block’s instance to a “stored” output. This field is only available for “stored” outputs, i.e., for an already “stored” input of an <output> block.
13. **Change initial value** : Allows modifying the initial value associated with the block’s instance of a “stored” output. This field is only available for “stored” outputs, i.e., for an already “stored” input of an <output> block.

14. **Delete initial value** : Allows removing the initial value associated with the block's instance of a "stored" output. This field is only available for "stored" outputs, i.e., for an already "stored" input of an <output> block.
15. **Override** : allows overwriting the type and value of the input inside the Target. This field is only available if Monitoring mode is activated.
16. **Properties**: displays the properties of the input like this.



- 4.2.4.c - Insert a block

To insert a block, you need to right-click on a virgin area, in order to display the following context menu:

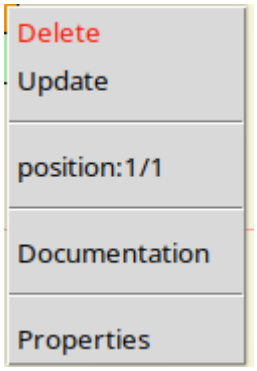


You then need to choose the type of block you want:

- Add input: to create a new entry in the **user** block currently being edited.
- Add output: to create a new output in the **user** block currently being edited.
- Add system bloc: for insert a block that is part of the standard library.
- Add user bloc: for insert a previously created **user** block.

- 4.2.4.d - Delete a block

To delete a block, you can right-click on the header of the block to be deleted, in order to display the following context menu:



Then select Delete.

You can also position the cursor on the header of the block to be deleted, then press the "Delete" key.

- 4.2.4.e - Create a link

To link the output of one block to the input of another block, click and hold the left mouse button on the output of the first block, then drag (left button still pressed) until the input of the second block, and then release the left mouse button. Links can be created from either input to output or output to input, but an input can only have one source (i.e., originate from only one output).

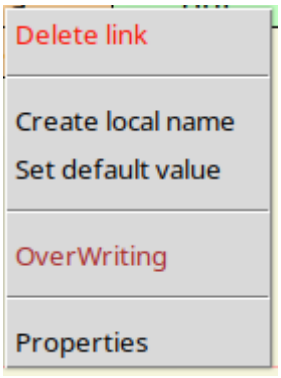
- 4.2.4.f - Delete a link

There are two ways to remove a link:

Either right-click on the link,in order to display the following context menu:

then select Delete link

Either right-click on the input located at the end of the link, in order to display the following context menu:



Then select Delete link

- 4.2.5 - Status bar

The status bar displays :

- The state of there connection with the Target.
- The position of the mouse in relation to the graphics area (canvas)).
- The zoom coefficient.
- The position of the mouse relative to the crosshair.

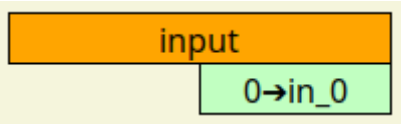
- 4.3 - input(s)/output(s)

The interface of a **user** block **i**s made up of input(s) and output(s); the latter are optional but essential for the interconnection with the other blocks.

- An inputs is collected using a **system** blocks named <input>
- An output is delivered using a **system** blocks named <output>

- 4.3.1 - <input> block

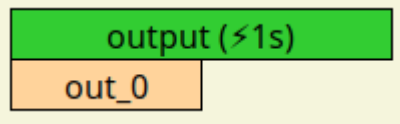
Here is the external interface of the <input> **system** block.



Its function is to define an input for a user block. The <input> block has no input, but possesses an output named "in_0", that is the variable (of any type) that will be entered into the user block. Its default value is 0 (int), and it can be adjusted for each instance of an <input> block. This output can be renamed; the new name will appear in the user block's external interface as an input.

- 4.3.2 - <output> block

Here is the external interface of the<output> **system** block.



Its function is to define an input for a user block. The <input> block has no input, but possesses an output named "in_0", that is the variable (of any type) that will be entered into the user block. Its default value is 0 (int), and it can be adjusted for each instance of an <input> block. This output can be renamed; the new name will appear in the user block's external interface as an input.

- 4.3.3 - External interface of a user block

Here is a **user** block called the <régle de 3> Which contains three <input> blocks and one <output> block.

- The output of the first <input> block has been named to "in"
- The output of the second <input> block has been named to "coeff"
- The output of the third block <input> has been named to"div"
- The input for the <output> block has been named to "out".

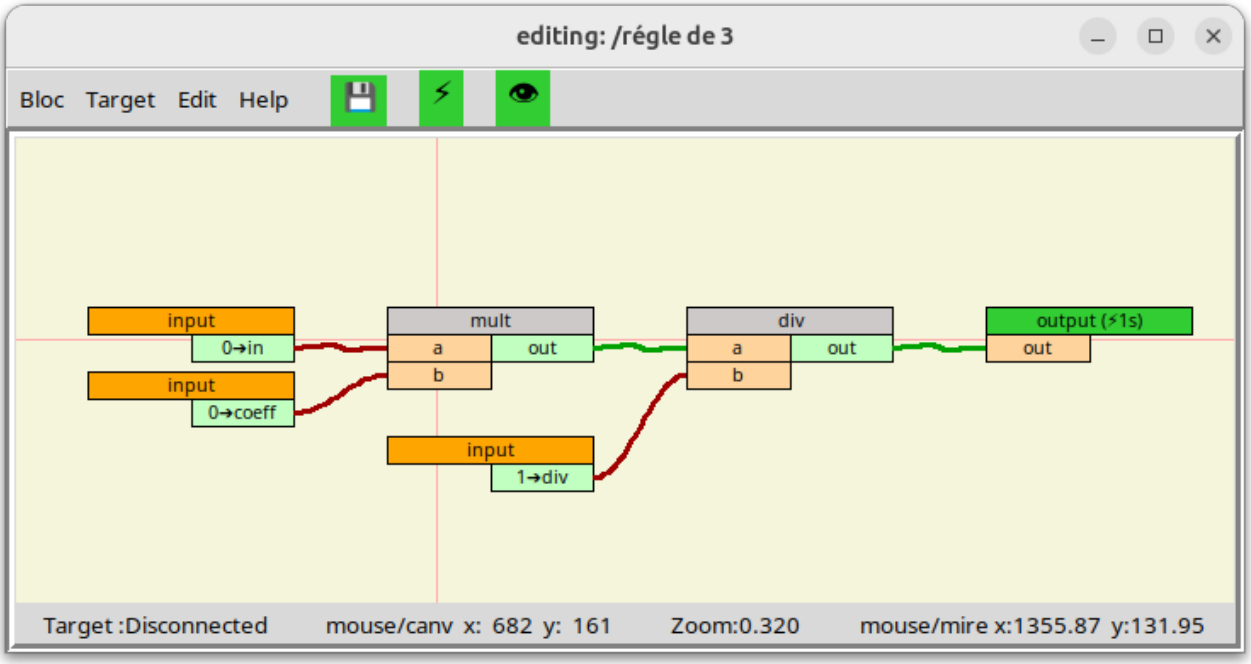
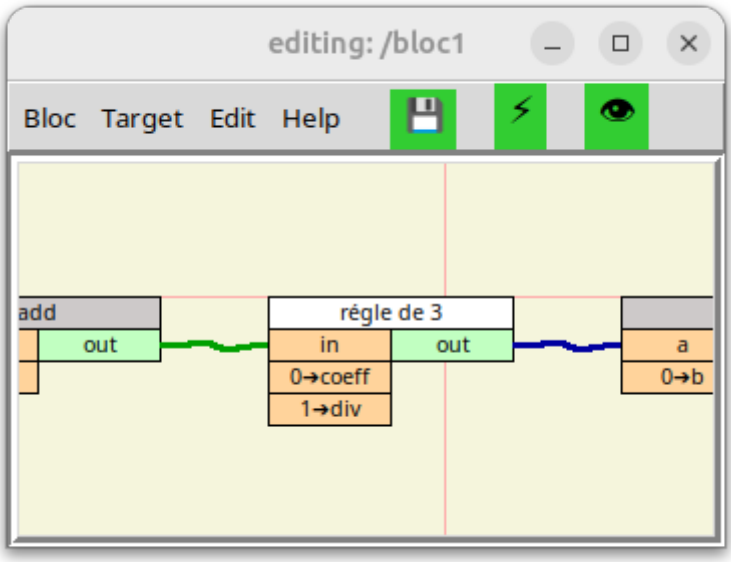


Figure: <régle de 3> **user** block

Here is the external interface of the <régle de 3> block, here a **user** block named <block1> refers to the <rule of 3> **user** block, the 3 inputs and the output correspond to the <input>/<output> defined in the <rule of 3> block and are indeed present in its external interface.



Figure; external interface of the block**user**<rule of 3>

The order in which inputs and outputs of appear in the external interface of a **user** block can be modified as follows:

1. Open the user block whose input and/or output order, by double-clicking on the header of the block to be modified, if it is present in the graphics area, or by using the menu bar (bloc/open) or (block/open new window then bloc/open).
2. Right-click on the header of the <input> or <output> block, depending on whether you want to change the order of the inputs or outputs. This will bring up the context menu. In this menu there is a field **Position: x/y** where “x” indicates the position of the input or output you have selected in the external interface of the block.
3. Then you can select **move up** or **move down** to shift it one a position up or down.

- 4.4 - Editing/Monitoring mode

Switching from "editing" mode to "monitoring” mode, and vice versa, is carried out either by clicking on the "eye" icon (👁), either by pressing the well button of the mouse, of course the block

being on screen must be in progress (Running) in the Target to be able to switch to "monitoring" mode.

- 4.4.1 - Editing Mode

This mode is dedicated to creating and modifying **user** blocks.

Here is an example of a **user** block which calculates the average of 2 numbers in “editing” mode, please note that window title shows the current mode:

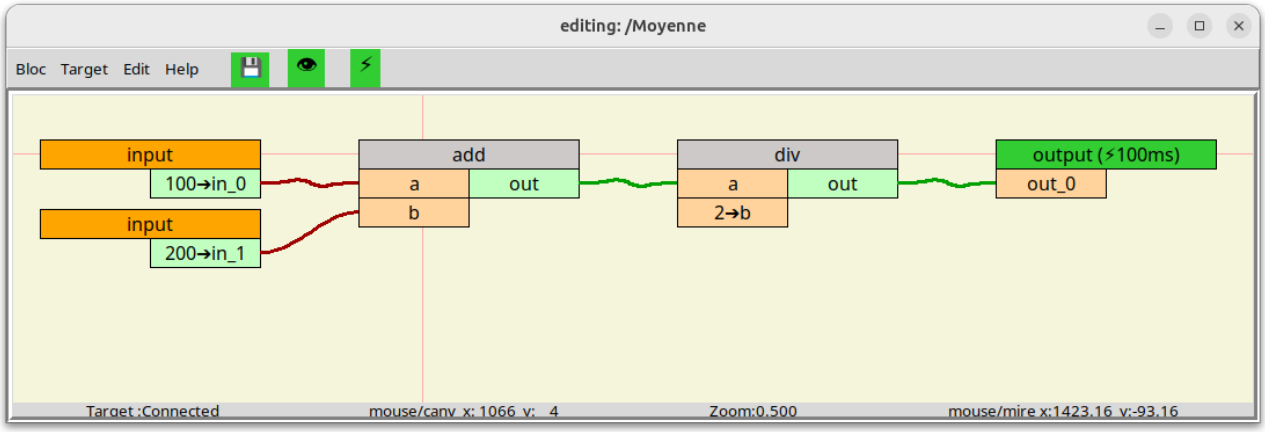


Figure 2:block in edit mode

- 4.4.2 - Monitoring Mode:

This mode is dedicated to debugging; it allows real-time visualization of the values of variables within a **user** block or a **user** sub-block**...**

The validity of each variable is represented by the background color:

- Green = valid
- Red = invalid
- Yellow = overridden and valid
- Black = overridden and invalid

Here is an example of <Moyenne> block in “Monitoring” mode (dynamic visualization), please note that window title shows the current mode:

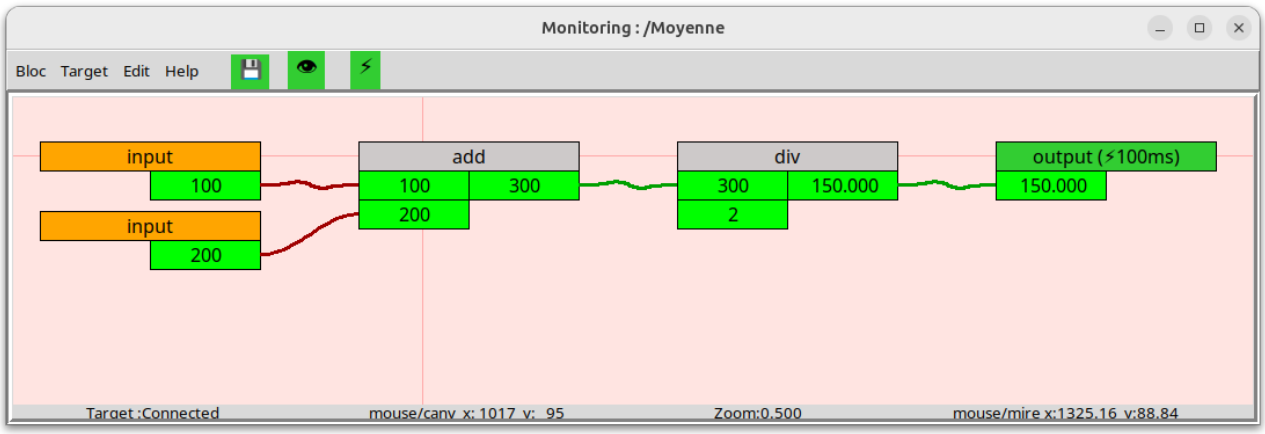


Figure3: blockin dynamic visualization mode

- 4.5 - Validity bit

The validity bits can be read or amended using the **system** blocks following:

- <valideRead>: allows you to read the validity bit, this allows for example select a fallback value in the case of variable is " invalid "
- <valideWrite>: allows writing the validity bit, authorizing the user to define its own equation of validity.

- 5 - Interpreter

- 5.1 - Generalities

Remember : The **Interpreter** is installed on a PC called " Target ", it executes the programs (**user** blocks) that control the industrial equipment using the remote I/O.

Some orders related to the blocks **user** already present in The **Interpreter,** such that as starting, stopping, initializing, swapping, and deleting, can be performed by the **Interpreter** through its integrated web server; see chapter "Web server"

The operations to modify a **user** block or to view in real time the values of the variables in execution (running) in the Target, are only available from the **Block editors**.

- 5.2 - Execution Method

For a chain of blocks contained in a **user** block to be executable, it is imperative that the chain ends with an <output> block and this last lock must be associated with an event (like the periodic event at 100ms For example). The **Interpreter** evaluates periodically all the blocks <output> of the a “Running” **user** block, thus triggering the evaluation of previous block which requires evaluation of the previous block and so on. This approach eliminates the need to explicitly define the execution order.

It is possible to include, within a single user block, multiple block chains that are executed at different intervals. In fact, a user block can contain several <output> blocks, each of which can be associated with a different periodic event. If there are links between block chains with different execution intervals, a system <previous> block must be inserted between each inter-chain connection, so that the chain intended to run more slowly is not executed at the same rate as the faster chain.

- 5.3 - Hot Swap

For on-the-fly loading to be possible, the **Interpreter** has for each **user** block (program) two versions; **Shift A a**nd **Shift B**. The new version of a **user** block is always downloaded into the **Shift** which is not currently running, when a "hot swap" is initiated, the current values of the “stored” variables of running **Shift** are transferred to the other **Shift**, which then takes over.

“Stored” variables are variables that must be retained between two execution cycles; they are outputs of **system** blocks such as <previous>, <memory>, <clock>, <edge>, <integrator>, etc.

- 5.4 - Validity bit

Generally speaking, as long as a block has not been evaluated, its outputs are "invalids»

The validity associated with each output is evaluated automatically during that the block is executed. If an inputs necessary for evaluating the output are "invalid" or if the exit of the block cannot be evaluated because of a type conflict or other the exit will be"invalid" Otherwise it will be "valid"

- 5.5 - HMI/SCADA interface

The **Interpreter** includes an OPC UA server; dedicated system blocks are available to let you expose the variables you witch to publish.

- opc_snode : create a node
- opc_sread : read a variable
- opc_swrite : write a variable

- 5.6 - Remote Input/Output

Remote I/O are handled by dedicated **system** blocks.

Currently, only the Modbus/TCP protocol is supported:

- modbus_conn: TCP connection with the remote I/O hardware module
- modbus_read: reading from a ModBus slave
- modbus_write: writing to a ModBus slave

- 5.7 - Raspberry Pi GPIO

The GPIO pins are managed by the following dedicated system blocks:

- **GPIO_DI**: configuration and reading of a digital input on a GPIO pin
- **GPIO_DO**: configuration and writing of a digital output on a GPIO pin
- **GPIO_PWM**: configuration and writing of a PWM output on a GPIO pin

- 5.8 - Create a new system block

For add a block has the standard library, it's necessary :

1. Create the external interface (graphical interface) of the new **system** block.
2. Code the associated "Python" function to the new **system** block and add the block name to the list of blocks in the standard library.

Note that it will be necessary to restart the **Executor a**nd the **Block editor** for this new **system** block to be taken into account**.**

- 5.8.1 - Create the external interface

Open the **Block editor**.

In the menu bar, click on "Bloc" then "Change Properties" to open the following "bloc info" window:

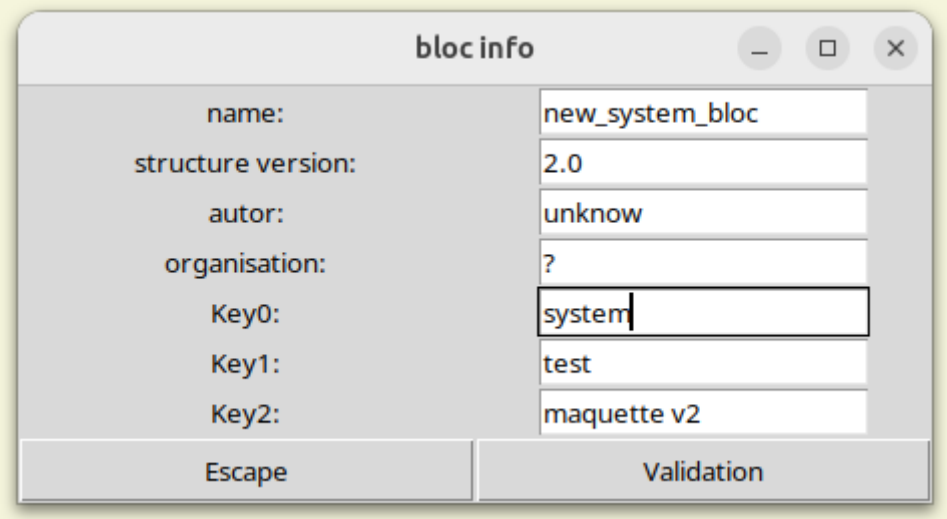


Figure: info bloc

Enter the name of the new **system** block in the "name:" field.

Enter the keyword "system" in the "Key0:" field.

Confirm your entry by clicking "Validation".

In the graphics area add **system** blocks<input> and/or <output> depending on the number of inputs and outputs needed, rename these inputs/outputs, assign a default value of inputs, optionally define some output as “stored” variables, then press the [F3] key to save the graphical interface of your new **system** block.

To verify that the interface has been created correctly, open a new window and then add the newly added **system** block**,** and verify that the external interface matches what you want.

- 5.8.2 - Create the Python code

- 5.8.2.a - Create a procedure dedicated to the new system block

The functions associated with the **system** block are in the file /Target/exec.py, by For example, here is the Python functionassociated withblock**system**<and>:

```
1  def c_exesubloc_and (pebloc, pieb, pio, pthread):
2      """ exécution du bloc a et b (dans la boucle récurcive)"""
3      cesubloc = pebloc.sublocs[pieb]
4      #print ("<AND> les paramètres sont: pieb=", pieb, ",   pio=", pio, ",   counter=", pthread['counter'])
5      if cesubloc.header['counter'] == pthread['counter']:
6          pass
7          #print ("<AND>", "  cesubloc['counter'] == pthread['counter']: =", pthread['counter'], "   (output[", pio, "] inchangée)")
8      else:
9          cesubloc.header['counter'] = pthread['counter']
10         try:
11             pebloc.c_exebloc_recup_inputs(pieb, pthread)
12             cesubloc.c_exesubloc_validation_standard()
13             cesubloc.outputs[0]['var'] = cesubloc.inputs[0]['var'] and cesubloc.inputs[1]['var']
14         except:
15             print ("<AND>", PARAM_TEXT_EXCEPTION)
16             for output in cesubloc.outputs:
17                 output['valide'] = False
18         cesubloc.c_exesubloc_overwriting_outputs()
19     #print ("<AND> retourne l'output [", pio, "]: var=", cesubloc.outputs[pio]['var'], "val=", cesubloc.outputs[pio]['valide'])
20     return cesubloc.outputs[pio]
```

Figure: “c_exesubloc_and” function

If we disregard the elements related to debugging, this is all that remains:

```
1  def c_exesubloc_and (pebloc, pieb, pio, pttread):
3          cesubloc = pebloc.sublocs[pieb]
5          if cesubloc.header['counter'] == pthread[‘pcounter’]:
6                  pass
8          else:
9                  cesubloc.header['counter'] = pthread[‘pcounter’]
10                 try:
11                         pebloc.c_exebloc_recup_inputs(pieb, pthread)
12                         cesubloc.c_exesubloc_validation_standard()
13                         cesubloc.outputs[0]['var'] = cesubloc.inputs[0]['var'] and cesubloc.inputs[1]['var']
14                 except:
16                         for  output in cesubloc.outputs :
17                                 output['valide'] = False
18                 cesubloc.c_exesubloc_overwriting_outputs()
20         return cesubloc.outputs[pio]
```

Explanation of the code:

Line 1: The name of the function is "c_exsubloc_" followed by system block name. The parameters are:

- pebloc: is the "executable" structure of the user block running.
- pieb: is the index of the system block to execute
- pio: is the index of the output whose value needs to be returned
- pthread: is the structure that contains all the tasks

Line 3: allows you to point to the block to be executed.

Line 5: checks if the block has already been executed

Line 9: marks the block as already executed

Line 11: recover all the block entries

Line 12: affects all bits validation of block outputs based on its inputs

Line 13: assigns the output based on the inputs (logical AND)

Lines 16&17: sets the validity bit to "invalid" in case the block calculation did not complete correctly

Line 20: returns the output requested (pio is the output index)

In most cases, your new function will be identical to that of the <and> **system** block except:

- line 1: since the function name will be different.
- line 13: since you will assign the outputs according to the inputs as needed.

- 5.8.2.b - Associate procedure and “block name” parameter

Still in the file: /Target/exec.py, you need to add the association between the name of your new **system** block and the newly created Python function, that takes place in the function “recup_procedure”.

Note that this function is also used by the **Block editor** when a **user** block is compiled:

```
def recup_procedure(psubloc):
    proc_name = "recupe_procedure"
    """ ajout l'adresse de la procédure qui correspond au nom du bloc"""
    if   psubloc.header['name'] == PARAM_NAME_BLOC_DT:           procedure= c_exesubloc_dt
    elif psubloc.header['name'] == PARAM_NAME_BLOC_OUTPUT:       procedure= c_exesubloc_output
    elif psubloc.header['name'] == PARAM_NAME_BLOC_INPUT:        procedure= c_exesubloc_input
    elif psubloc.header['name'] == PARAM_NAME_BLOC_PREVIOUS:     procedure= c_exesubloc_previous
    elif psubloc.header['name'] == PARAM_NAME_BLOC_SELECT:       procedure= c_exesubloc_select
    elif psubloc.header['name'] == PARAM_NAME_BLOC_VALIDREAD:    procedure= c_exesubloc_validRead
    elif psubloc.header['name'] == PARAM_NAME_BLOC_VALIDWRITE:   procedure= c_exesubloc_validWrite
    elif psubloc.header['name'] == PARAM_NAME_BLOC_COMP:         procedure= c_exesubloc_comp
    elif psubloc.header['name'] == PARAM_NAME_BLOC_AND:          procedure= c_exesubloc_and
    elif psubloc.header['name'] == PARAM_NAME_BLOC_OR:           procedure= c_exesubloc_or
    elif psubloc.header['name'] == PARAM_NAME_BLOC_NOT:          procedure= c_exesubloc_not
    elif psubloc.header['name'] == PARAM_NAME_BLOC_EDGE:         procedure= c_exesubloc_edge
    elif psubloc.header['name'] == PARAM_NAME_BLOC_CABLIN:       procedure= c_exesubloc_cablin
    elif psubloc.header['name'] == PARAM_NAME_BLOC_CABLOUT:      procedure= c_exesubloc_cablout
    elif psubloc.header['name'] == PARAM_NAME_BLOC_ADD:          procedure= c_exesubloc_add
    elif psubloc.header['name'] == PARAM_NAME_BLOC_SUB:          procedure= c_exesubloc_sub
    elif psubloc.header['name'] == PARAM_NAME_BLOC_MULT:         procedure= c_exesubloc_mult
    elif psubloc.header['name'] == PARAM_NAME_BLOC_DIV:          procedure= c_exesubloc_div
    elif psubloc.header['name'] == PARAM_NAME_BLOC_MINMAX:       procedure= c_exesubloc_minmax
    elif psubloc.header['name'] == PARAM_NAME_BLOC_CONST_PI:     procedure= c_exesubloc_const_pi
    else:
        print (proc_name, " ERROR: function not defined for this bloc <"+psubloc.header['name']+">")
    return procedure
```

Figure: function "recup_procedure"

To do this, you will need to add a line before the "else" and create a new parameter whose name will consist of:

"PARAM_NAME_BLOCK_" followed by the name of your new **system** block in capital letters.

- 5.8.2.c - Define the new system block file name

You will also need to modify the file /Target/PARAM_NAME_BLOC.py, in add to it a line to define the constant "PARARM_NAME_BLOCK_*yourblockname, t*he string of characters to the right of the "=" sign corresponds to the filename of your new **system** block, its extension will always be “.bloc”**.**

```
# ATTENTION: la source de ce fichier ce trouve dans le répertoire "Target"

# nom des blocs "system"
PARAM_NAME_BLOC_DT = "dt"
PARAM_NAME_BLOC_OUTPUT = "output"
PARAM_NAME_BLOC_INPUT = "input"
PARAM_NAME_BLOC_PREVIOUS = "previous"
PARAM_NAME_BLOC_SELECT = "select"
PARAM_NAME_BLOC_VALIDREAD = "validRead"
PARAM_NAME_BLOC_VALIDWRITE = "validWrite"
PARAM_NAME_BLOC_MINMAX = "minmax"
PARAM_NAME_BLOC_COMP = "comp"
PARAM_NAME_BLOC_AND = "and"
PARAM_NAME_BLOC_OR = "or"
PARAM_NAME_BLOC_NOT = "not"
PARAM_NAME_BLOC_EDGE = "edge"
PARAM_NAME_BLOC_ADD = "add"
PARAM_NAME_BLOC_SUB = "sub"
PARAM_NAME_BLOC_MULT = "mult"
PARAM_NAME_BLOC_DIV = "div"
PARAM_NAME_BLOC_CABLIN = "cablin"
PARAM_NAME_BLOC_CABLOUT = "cablout"
PARAM_NAME_BLOC_CONST_PI = "const.pi"

```

Figure: PARAM_NAME_BLOCK.py file

- 5.9 - Web Server

- 5.9.1 - Generalities

A HTTP server is integrated into the **Interpreter,** it allows the user through a simple web browser to view the status of the Target:

- View the available tasks and the number of <output> associated
- View overall and per task CPU load
- View the list of blocks available in the Target and the status (Pending, Ready, Running) of the **Shift**s **(**A & B)
- Visualize the state and value of the <output> of all blocks for the **Shift**s (A & B)
- Visualized the current Ethernet connections managed by the **Interpreter**

It also allows you to place simple commands such as:

- Stop the execution of a **user** block (order :Stop)
- Start the execution of a **user** block (command: Run)
- Initialize a **user** block (command: Initialize)
- Delete a **user** block (command: Delete)
- Swap the **Shift A a**nd **Shift B**
  - When none **Shift** is "Running" (command: ClodSwap)
  - When a **Shift** is "Running" (command: HotSwap)
- Launch command dedicated to debugging http
  - Printing all blocs known to the **Interpreter** (command: print list_compiled)
  - Printing all tasks known to the **Interpreter** (command: print list_threads)

- 5.9.2 - Web server address

By default, the address is: http://127.0.0.1:8080/
As in the evaluation version (downloadable from Github), since the target PC and the development PC are on the same PC, the web server's IP address is 127.0.0.1 (note that port 80, the default port for the HTTP protocol, is not used); we use port 8080 so that the Executor can be launched without needing a `sudo` command.

- 5.9.3 - Main menu

Menu
Bloc Output Overriding Task & Load Connection print list_compiled print list_threads

- Bloc : lists the blocks known to the **Interpreter**, show their status, and offers commands to execute based on their state.
- Output : lists all outputs known to the **Interpreter**, indicating their value and validity
- Override : lists all overrides known to the **Interpreter**, indicating their value and validity
- Task & load : lists all tasks known to the **Interpreter**, indicating their load
- Connetion : lists clients connected to the **Interpreter**,
- print list_compiled : This command prints all blocks known to the **Interpreter** (for debugging)
- print list_threads : This command prints all tasks known to the **Interpreter**(for debugging)

- 5.9.4 - Web pages example

- 5.9.4.a - *"Bloc" page*

bloc list			
bloc name	**status**		**order**
	shift A	**shift B**	
loop	Running	...	Stop Initialize
maskStart	Pending	Running	Stop Initialize HotSwap

Refresh

- 5.9.4.b - “Output” page

bloc output list											
bloc				**task**			**output**				
name	**shift**	**building time (yyyy/mm/dd)**	**status**	**name**	**id**	**exec order**	**name**	**id**	**type**	**validity**	**value**
loop	A	2026/04/30 - 14:05:17	Running	1s	2	0	Oloop	4	int	😊	13028
maskStart	A	2026/04/30 - 16:12:26	Down	1s	2	1	pp	8	...	...	...
maskStart	B	2026/04/30 - 16:12:38	Running	1s	2	2	pp	8	bool	😊	True

Refresh

- 5.9.4.c - "Overriding" page

Overriding list							
bloc			**io**				
main	**shift**	**path/name**	**name**	**type**	**id**	**value**	**validity**
maskStart	B	maskStart/(6)delay	delay	input	3	1.1	😊

Refresh

- 5.9.4.d - "Task & load" page

Task list								
Setting			**Feedback**					
name	**id**	**period**	**cycle time**	**cycle time min**	**cycle time max**	**counter**	**number of output**	**CPU load**
10s	0	10s	9.999883s	9.999883s	9.999883s	745	0	0.000%
3s	1	3s	2.999850s	2.999850s	2.999850s	2483	0	0.000%
1s	2	1s	0.999933s	0.999933s	1.000002s	7451	2	0.009%
200ms	3	200ms	200.260ms	199.924ms	200.408ms	37251	0	0.003%
100ms	4	100ms	99.853ms	99.838ms	99.992ms	74493	0	0.005%
50ms	5	50ms	50.244ms	49.791ms	50.359ms	148968	0	0.014%
20ms	6	20ms	19.880ms	19.786ms	20.401ms	372509	0	0.019%
10ms	7	10ms	10.350ms	9.806ms	10.384ms	745270	0	0.069%
5ms	8	5ms	4.883ms	4.780ms	5.380ms	1491506	0	0.054%
2ms	9	2ms	1.889ms	1.718ms	2.362ms	3829625	0	0.126%

User tasks CPU load = 0.30%
Refresh

- 5.9.4.e - "Connection" page

HTTP protocol			
...	**@ip**	**port**	**Host name**
Server	192.168.5.58	8080	jbar-HLYL-WXX9
Last Client	127.0.0.1	59368	Localhost

TCP/IP protocol	
...	**@ip**
server	192.168.5.58
client[0]	127.0.0.1

Refresh